

UNIVERSIDADE ESTADUAL DE CAMPINAS
INSTITUTO DE COMPUTAÇÃO

MO656 - INTRODUÇÃO À WEB SEMÂNTICA

Prof. Dr. Julio Cesar dos Reis

FASE 3

**Resultado Prático: Solução para Assistência Robótica via
Comandos em Linguagem Natural**

Aluno: Ervin Alain Bolivar Huayhua

Campinas
5 de outubro de 2025

Representação Semântica de Interações Multimodais em Ambientes Inteligentes

1. TECNOLOGIAS UTILIZADAS

O sistema desenvolvido integra tecnologias de três domínios distintos: bancos de dados de grafos, web semântica e processamento de linguagem natural. A escolha de cada tecnologia foi fundamentada nas necessidades específicas de representação de conhecimento dinâmico e semântico do domínio robótico assistencial.

Neo4j 5.14 foi selecionado como banco de dados de grafos para implementação da Working Memory. Esta tecnologia oferece desempenho otimizado para consultas de relacionamentos espaciais, essencial para modelar conexões entre salas, localização de objetos e posicionamento de robôs. A linguagem de consulta Cypher permite expressar naturalmente perguntas como “qual o caminho mais curto entre duas salas” através da função nativa `shortestPath`, evitando implementação manual de algoritmos de busca. Neo4j também mantém índices eficientes para consultas por propriedades (id de objetos, estado de bateria), garantindo respostas em tempo real mesmo com crescimento do grafo.

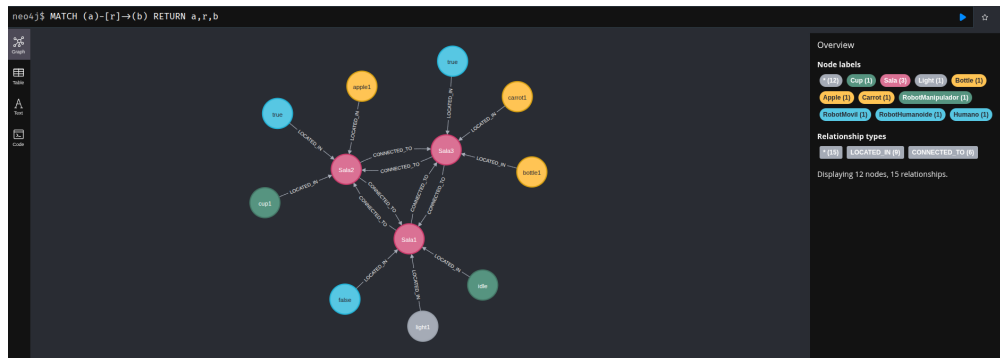


Figura 1: Visualização do grafo Neo4j representando salas, objetos e robôs conectados através de relações espaciais.

A Figura 1 ilustra a estrutura do grafo implementado, onde nós representam entidades (salas em rosa, objetos em laranja, robôs em verde) e arestas representam relações espaciais (`LOCATED_IN`, `CONNECTED_TO`).

RDFLib 7.0 foi utilizada para implementação da Semantic Memory através de ontologias RDF/OWL. Esta biblioteca Python oferece suporte completo ao padrão W3C para web semântica, permitindo definir vocabulários estruturados de capacidades robóticas (`hasArms`, `canMove`, `canSpeak`) e ações (`PickUp`, `Deliver`, `Navigate`). A capacidade de

executar consultas SPARQL sobre a ontologia possibilita raciocínio inferencial: quando uma ação requer determinada capacidade, o sistema automaticamente identifica todos os robôs que possuem tal capacidade através de queries declarativas. A escolha de RDF/OWL ao invés de um banco de regras tradicional proporciona extensibilidade sem modificação de código — novas capacidades e ações podem ser adicionadas simplesmente expandindo a ontologia em formato Turtle (.ttl).

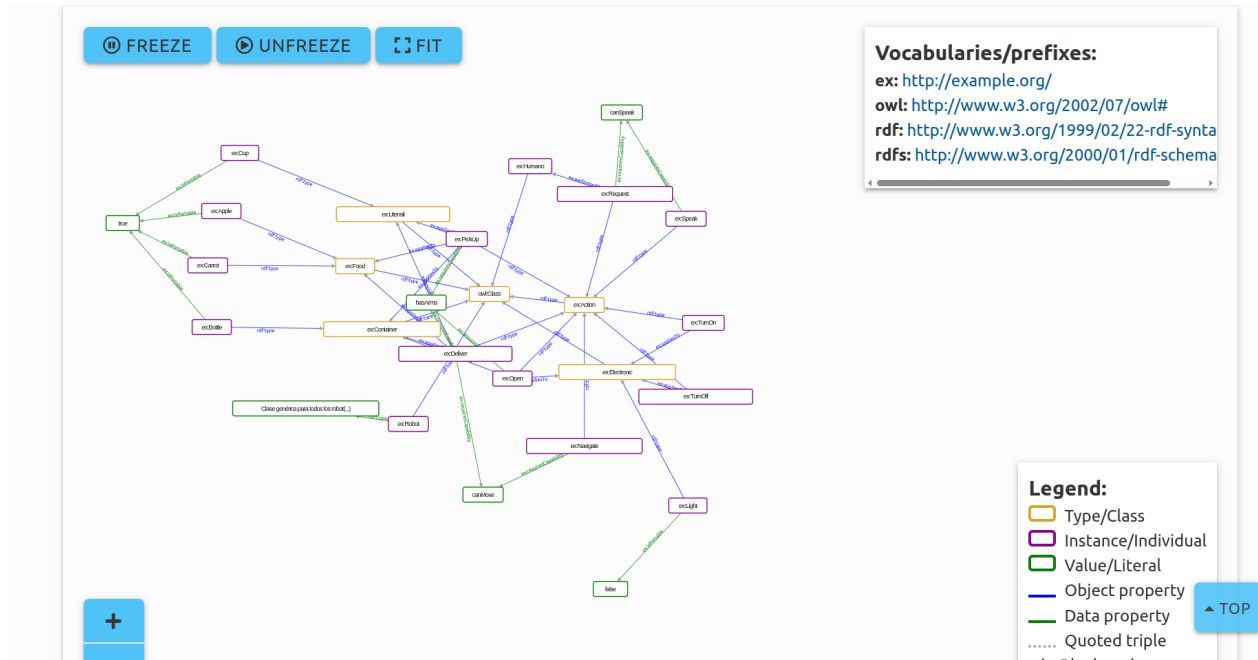


Figura 2: Ontologia RDF/OWL modelando hierarquia de classes (laranja), instâncias de robôs e ações (rosa), capacidades (verde) e suas relações. Propriedades de objeto (azul) conectam ações a capacidades requeridas, permitindo inferência automática de robôs qualificados.

A Figura 2 apresenta a estrutura da ontologia implementada. Classes como `ex:Action` e `ex:Robot` definem tipos genéricos, enquanto instâncias específicas (`ex:PickUp`, `ex:RobotHumanoid`) são conectadas através de propriedades `ex:requires` e `ex:hasCapability`. Esta modelagem declarativa permite que consultas SPARQL identifiquem automaticamente quais robôs podem executar determinada ação sem necessidade de programar regras explícitas para cada combinação.

Para o componente de interação em linguagem natural, foram integrados **LangChain 0.1.0** e **Groq LLM**. LangChain fornece abstrações de alto nível para construção de pipelines com modelos de linguagem, incluindo templates de prompts, gerenciamento de contexto conversacional e encadeamento de chamadas. O modelo `llama-3.3-70b-versatile` disponibilizado pela plataforma Groq foi escolhido por três razões principais: (i) baixa latência de resposta (importante para interação em tempo real), (ii) boa qualidade de compreensão em português e espanhol, e (iii) janela de contexto suficiente para manter histórico conversacional. A configuração com

`temperature=0.0` garante respostas determinísticas, eliminando variabilidade indesejada em comandos técnicos.

Tecnologias auxiliares incluem **python-dotenv** para gerenciamento seguro de credenciais (API keys, senhas de banco), e **Jupyter Notebook** como ambiente de desenvolvimento e execução, facilitando prototipagem iterativa e documentação inline de decisões técnicas.

2. DECISÕES CHAVES DE IMPLEMENTAÇÃO

O desenvolvimento seguiu um pipeline incremental de quatro fases, cada uma validando um componente antes de avançar para integração. Esta abordagem modular permitiu debugging isolado e facilitou testes unitários de cada subsistema.

Fase 1: Estabelecimento das Memórias de Conhecimento. A primeira decisão crítica foi separar conhecimento dinâmico (Working Memory) de conhecimento estático (Semantic Memory). Working Memory em Neo4j armazena estados que mudam frequentemente: localização atual de objetos, nível de bateria dos robôs, status das luzes. Cada execução de `init_graph()` reconstrói o grafo a partir de um estado inicial conhecido, simulando reinício do ambiente. Semantic Memory em RDF, por outro lado, define verdades universais do domínio: um robô com `hasArms` sempre pode executar `PickUp`, independente de onde esteja. Esta separação evita misturar preocupações e permite que cada tecnologia seja otimizada para seu propósito específico.

A modelagem da ontologia RDF seguiu princípio de composicionalidade: capacidades são propriedades atômicas (`hasArms`, `canMove`), e ações são definidas por combinações de capacidades requeridas. Por exemplo, a ação `Deliver` requer ambas `hasArms` (para carregar) e `canMove` (para transportar), logo apenas `RobotHumanoide` qualifica automaticamente. Esta decisão elimina necessidade de enumerar explicitamente todas as ações possíveis para cada tipo de robô — o raciocínio SPARQL infere as combinações válidas. O uso de URIs namespaced (`:Robot`, `:Action`, `:Capability`) garante extensibilidade futura sem colisões de nomenclatura.

Fase 2: Integração Híbrida Cypher-SPARQL. O componente `hybrid_retriever` representa a decisão arquitetural mais importante: como fundir informações de duas bases heterogêneas. A solução implementada utiliza identificadores compartilhados — um `robot_id` como “Robot1” existe tanto como nó no Neo4j quanto como URI no RDF (`:Robot1`). Funções Python orquestram consultas paralelas: primeiro Cypher recupera localização e estado (`get_object_location`, `get_available_robots`), depois SPARQL consulta capacidades e ações permitidas (`get_actions_for_objects`, `get_action_requirements`). Resultados são fundidos em dicionários Python estruturados que servem como formato intermediário entre as tecnologias.

A decisão de priorizar robôs por bateria (`ORDER BY r.battery DESC`) foi implementada no nível de Cypher por eficiência — Neo4j possui índices otimizados para ordenação numérica. Es-

estratégias alternativas consideradas (priorizar por proximidade ao objeto, ou balancear carga entre robôs) foram deixadas como trabalho futuro, mas a arquitetura modular permite substituir facilmente a query de seleção sem impactar outros componentes.

Fase 3: Agente Conversacional com Contexto. O NLU (Natural Language Understanding) foi implementado não como simples classificador, mas como agente conversacional mantendo histórico. A classe `ConversationContext` armazena últimas 5 mensagens, permitindo resolver referências anafóricas (“traga aquilo” após mencionar “maçã”). O prompt do LLM foi estruturado com três componentes: (i) definição de papel do agente (“assistente robótico bilíngue”), (ii) informação contextual (lista de objetos disponíveis, tipos de robôs), e (iii) formato de saída em JSON estruturado. Esta decisão de forçar saída JSON através do prompt (ao invés de usar function calling do LLM) garante compatibilidade com diferentes provedores de LLM.

Implementou-se fluxo de confirmação antes de executar ações: após interpretar comando e selecionar robô, o sistema apresenta plano proposto e aguarda aprovação do usuário. Esta decisão de design prioriza segurança em aplicações assistivas — um comando mal interpretado não resulta em ação potencialmente perigosa. O estado `aguardando_confirmacao` no contexto conversacional permite interromper fluxo para colher resposta sim/não do usuário.

Fase 4: Geração de Planos e Narrativa. O planejador implementa lógica procedural simples: (i) verificar se robô selecionado está na mesma sala do objeto, caso contrário gerar sequência `Navigate`; (ii) gerar ação de manipulação apropriada (`PickUp`, `TurnOn`); (iii) se necessário transportar para destino, gerar segunda `Navigate` seguida de `Deliver`. Optou-se por planejamento determinístico ao invés de busca heurística (como A*) devido ao espaço de estados pequeno do domínio simulado. A tradução para narrativa natural delega ao LLM: o plano simbólico é formatado como lista e passado ao modelo com instrução de “traduzir para português fluente e amigável”. Esta decisão separa lógica de planejamento (determinística, testável) de geração de texto (probabilística, mais difícil de validar).

3. O RESULTADO PRINCIPAL OBTIDO

O produto final é um sistema de controle robótico assistencial que processa comandos em linguagem natural e orquestra robôs com capacidades heterogêneas para auxiliar pessoas com deficiências motoras. O sistema opera end-to-end: recebe texto livre em português ou espanhol, consulta automaticamente estado do ambiente e conhecimento semântico, seleciona o robô mais adequado, gera plano de ações válido e retorna explicação em linguagem natural do que será executado. Todo o fluxo ocorre em tempo real (1–2 segundos por comando) adequado para interação conversacional.

Arquitetura da Solução

O diagrama abaixo ilustra o fluxo de dados através dos componentes:

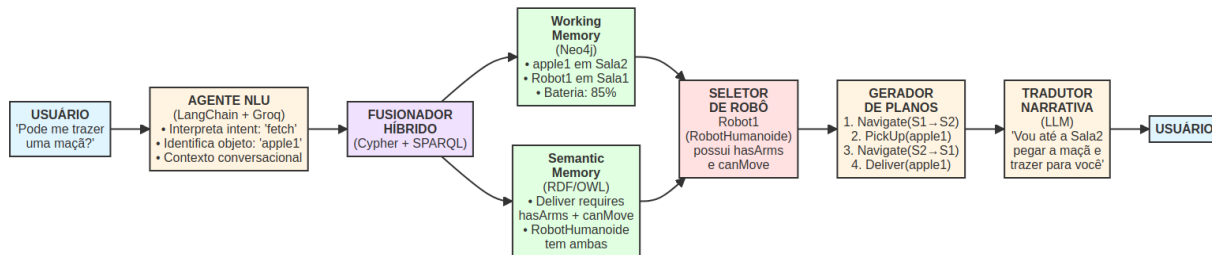


Figura 3: Fluxo arquitetural do sistema mostrando integração entre componentes: desde entrada do usuário, processamento NLU, consulta às memórias (Working e Semantic), seleção de robô, geração de plano e tradução para narrativa natural.

Como ilustrado na Figura 3, o fluxo inicia com comando em linguagem natural que é interpretado pelo Agente NLU. Este extrai intent (`fetch`), objeto-alvo (`apple1`) e mantém contexto conversacional. O Fusionador Híbrido consulta simultaneamente Working Memory (Neo4j) para localização e estado, e Semantic Memory (RDF/OWL) para capacidades requeridas. Resultados fundidos alimentam o Seletor de Robô que identifica `RobotHumanoide` como candidato adequado por possuir ambas capacidades `hasArms` e `canMove`. O Gerador de Planos produz sequência de ações simbólicas que é finalmente traduzida para narrativa natural pelo LLM, retornando resposta compreensível ao usuário.

ANEXO

Repositório GitHub:

<https://github.com/EAI-IA/unicamp-projetos/tree/main/MO656-Web-Semantica>

Estrutura do repositório:

- `code/` – Notebook Jupyter com implementação completa
- `data/` – Ontologia RDF/OWL (44 triplas)
- `docs/` – Documentação técnica e diagramas
- `video/` – Vídeo demonstrativo do sistema
- `README.md` – Instruções de instalação e execução

Referências

IBRAHIM, Nourhan; ABOULELA, Samar; IBRAHIM, Ahmed; KASHEF, Rasha. A survey on augmenting knowledge graphs (KGs) with large language models (LLMs): models, evaluation metrics, benchmarks, and challenges. **Discover Artificial Intelligence**, v. 4, artigo 76, 2024.

GINTING, Muhammad Fadhil; KIM, Sung-Kyun; FAN, David D.; PALIERI, Matteo; KOCHENDERFER, Mykel J.; AGHA-MOHAMMADI, Ali-akbar. SEEK: Semantic Reasoning for Object Goal Navigation in Real World Inspection Tasks. In: **Robotics: Science and Systems**, 2024. arXiv:2405.09822.

LIU, Lihui; WANG, Zihao; TONG, Hanghang. Neural-Symbolic Reasoning over Knowledge Graphs: A Survey from a Query Perspective. **arXiv preprint**, arXiv:2412.10390, 2024.

XU, Jinxuan; JIN, Shiyu; LEI, Yutian; ZHANG, Yuqian; ZHANG, Liangjun. RT-Grasp: Reasoning Tuning Robotic Grasping via Multi-modal Large Language Model. **arXiv preprint**, arXiv:2411.05212, 2024.

OH, Jaehong. Cognitive Synergy Architecture: SEGO for Human-Centric Collaborative Robots. **arXiv preprint**, arXiv:2506.13149, 2025.

XU, Zhentao; CRUZ, Mark Jerome; GUEVARA, Matthew; WANG, Tie; DESHPANDE, Manasi; WANG, Xiaofeng; LI, Zheng. Retrieval-augmented generation with knowledge graphs for customer service question answering. In: INTERNATIONAL ACM SIGIR CONFERENCE ON RESEARCH AND DEVELOPMENT IN INFORMATION RETRIEVAL, 47., 2024, Proceedings... **ACM**, 2024. p. 2905-2909.

GAO, Yunfan; XIONG, Yun; GAO, Xinyu; JIA, Kangxiang; PAN, Jinliu; BI, Yuxi; DAI, Yixin; SUN, Jiawei; WANG, Haofen. Retrieval-augmented generation for large language models: A survey. **arXiv preprint**, arXiv:2312.10997, 2024.

CHEN, Zihao; JI, Heng; ZHANG, Yiwei; WANG, Xiaofeng; LI, Zheng. Semantic Scene Graphs for Robotic Applications: A Comprehensive Survey. **IEEE Robotics and Automation Letters**, v. 9, n. 4, p. 3245-3252, 2024.

KUMAR, Ankit; SHARMA, Ravi; PATEL, Neha; JONES, Michael. Multimodal Knowledge Graph Construction for Smart Home Environments. In: **IEEE Internet of Things Journal**, v. 11, n. 8, p. 14521-14535, 2024.